TECHNICAL ARCHITECTURE REFERENCE

PenguinShot 핵심 서브시스템 구조 명세서

Enemy(적), Object Storage(오브젝트 보관소), Placement(필드 배치) 아키텍처 및 라이프사이클 분석

대상 프로젝트	분석 범위	작성 일시	문서 버전
PenguinShot (Unity)	Enemy, Storage, Placement	2026년 9월	v1.0.0 (Release)

0 시스템 개요 및 3대 핵심 모듈 관계

본 문서는 **PenguinShot**의 전투와 필드 상호작용을 구성하는 3대 핵심 축인 `Enemy`, `Object System`, `Placement System`의 클래스 구조와 동작 라이프사이클을 체계적으로 정리합니다.

- **Enemy System** → **적 처치 (OnEnemyKilledByPlayer)**: 적 사망 시 `ObjectDropManager`에 신호 전달
- **Object System** → **동적 확률 드랍 & 즉시 입고**: `ObjectStorageManager`에 개별 `ObjectItem` 즉시 적재
- **Placement System** → **정비시간 청사진 배치 & 실물화**: 슬롯 클릭 → 청사진 검증 → 확정 시 보관소 차감

1 Enemy (적) 시스템 구조

적 시스템은 **기본 스탯 및 이벤트 제어**, **스플라인 경로 추적 및 FSM(유한 상태 머신) AI**, **핀볼 물리 충돌 반응**, **웨이브 스케줄러**로 구성됩니다.

1.1 핵심 클래스 명세

클래스명	파일 경로	책임 및 주요 역할
`Enemy`	`Assets/Game/Scripts/Enemy`	모든 적의 베이스 클래스. `MaxHealth`, `CurrentHealth`, `BaseGold`, `TakeDamage()`, `Die()`, `ReachBase()` 등 공통 스탯 및 생명주기 이벤트 관리.
`NormalEnemyAI`	`Assets/Game/Scripts/Enemy`	지상 적의 기본 AI 클래스 (`Enemy` 상속). 내부 FSM(`EnemyStateMachine`)을 보유하여 이동/넉백/복귀를 제어. **플레이어 공격으로 처치 시 `ObjectDropManager.OnEnemyKilledByPlayer()` 호출, 베이스 도달 시 `GameManager.DecreaseLife()` 및 `ObjectDropManager.OnEnemyReachedBaseOrDespawned()` 호출.**
`EnemyStateMachine`	`Assets/Game/Scripts/Enemy/AI`	상태 패턴 기반 FSM 엔진. `IEnemyState` 인터페이스(Enter, Update, FixedUpdate, Exit, Collision)를 통해 적의 행태 전이를 캡슐화.

클래스명	파일 경로	책임 및 주요 역할
`CatmullRomPath`	`Assets/Game/Scripts/Enemy`	웨이포인트를 부드러운 스플라인 곡선으로 보간. 진행률 (`pathProgress`)에 따라 목표 월드 좌표 및 접선 벡터(회전각) 제공.
`EnemyBounceSurface`	`Assets/Game/Scripts/Physics`	`PinballBounceSurface` 상속. 펭귄 충돌 시 콤보 누적, `CombatResolver` 를 통한 피해 계산 및 적 넉백 트리거.
`EnemyManager`	`Assets/Game/Scripts/Enemy`	적 관리 총괄 싱글톤. 웨이브 데이터(`WaveData` , `EnemySpawnEntry`) 기반 코루틴 스폰, 적 처치 시 코인 인스턴스화, **정비 시간에 다음 웨이브 이동 경로를 가늠선(LineRenderer)으로 렌더링**.

1.2 FSM (상태 머신) 세부 동작 구조

EnemyPathFollowState (경로 추적)

- `CatmullRomPath` 를 따라 `moveSpeed` 기반 진행률 누적
- 스플라인 접선 방향(Tangent)을 바라보도록 회전 보간
- 경로 종점 도달 시 `OnReachPathEnd()` 실행

EnemyKnockbackState (피격 넉백)

- 펭귄 충돌 임펄스에 비례한 거리 계산 (Ease-out 감속 곡선)
- 넉백 거리의 10%만큼 경로 진행도(`pathProgress`) 후퇴 손실
- 넉백 종료 후 즉시 복귀 상태로 전이

EnemyReturnToSplineState (궤적 복귀)

- 넉백으로 밀려난 이탈 좌표에서 현재 목표 스플라인 지점으로 보간 이동
- `returnToSplineSpeedMultiplier` (1.5배) 속도로 신속 복귀
- 도달 시 다시 `EnemyPathFollowState` 로 원복

EnemyStunState (기절 상태)

- 지정된 `duration` 동안 이동 및 물리 연산 정지
- 기절 지속 시간 만료 시 이전 상태 또는 복귀 상태로 전환

1.3 특수 기믹 적 (Special Enemy Types)

- **ShrinkEnemyAI**: `NormalEnemyAI` 상속. 펭귄과 3회 충돌 누적 시 대상 펭귄을 5초간 50% 크기로 축소(Shrink)시켜 피격 면적과 파괴력을 감소시킴. 축소 상태에서 외부 재충돌 시 즉시 해제.
- **ComboResetEnemyAI**: `NormalEnemyAI` 상속. 5초 주기로 반경 내 최대 3마리의 펭귄에게 광역 효과 발동. 펭귄의 누적 콤보 0 초기화 및 과부하 강제 해제 후 0.5초 동안 속도를 0으로 정지(Freeze)시킴.
- **SeagullAI**: 공중 무적 적 (HP 999,999). [비행 ➔ 펭귄 탐색/추적 ➔ 다이빙 포획 ➔ 플리퍼 방향 투척 ➔ 지상 휴식 ➔ 맵 밖 퇴장]의 5단계 고유 상태 머신 운영.

2 Object 및 Object 보관소(인벤토리) 구조

오브젝트 시스템(`Game.ObjectSystem`)은 게임 진행 중 필드에 설치하는 오브젝트(Bumper, Slingshot 등)를 수집, 저장, 관리하는 전용 인벤토리 파이프라인입니다.

> ⚠️ **중요: 프로젝트 내 인벤토리 시스템 이원화 구분**
>
> • **ObjectStorageManager (오브젝트 보관소)**: 필드에 직접 배치하는 설치물(Bumper, Slingshot 등) 전용 인벤토리. 웨이브 중 적 처치로 드랍되며 맵 전환 시 초기화됩니다.
> • **ItemInventoryManager (아이템 인벤토리)**: 펭귄 장착용 장비(`EquipmentSO`), 상점 부적(`ShopItemBase`), 오브젝트용 각인(`EngravingItem`)을 보관하는 일반 RPG 인벤토리입니다.

2.1 핵심 클래스 명세

클래스명	네임스페이스 / 위치	책임 및 주요 역할
`ObjectItem`	`Game.ObjectSystem`	보관소에 저장되는 개별 오브젝트 데이터. `InstanceId` (GUID 고유 식별자)와 `ObjectType` (Bumper, Slingshot, CloneGate)을 보유. 수량 스택이 아닌 **개별 인스턴스(Non-stackable)**로 관리.
`ObjectStorageManager`	`Game.ObjectSystem` (Managers)	**오브젝트 보관소 총괄 싱글톤**. `AddObject()` , `RemoveObject()` 제공. 보관 한도 무제한. **맵 전환(`SceneManager.sceneLoaded`) 시 `ClearOnMapTransition()` 을 실행하여 보관소 잔여분 및 필드 설치물을 전량 파괴**.
`ObjectDropManager`	`Global / Managers`	**동적 확률 기반 드랍 제어 싱글톤**. 웨이브 시작 시 목표 드랍량(`targetDropCount`)과 총 적 수를 설정. 적 처치 시마다 잔여 적 대비 남은 드랍 목표로 확률을 동적 갱신하여 확정적 분배. 필드 바닥에 떨어지지 않고 **보관소로 즉시 직행 입고**.
`ObjectStorageUI`	`Game.ObjectSystem.UI` (Managers)	보관소 UI 패널. `ObjectStorageManager` 의 이벤트(`OnObjectAdded` , `OnObjectRemoved` , `OnStorageCleared`)를 실시간 구독하여 슬롯을 동적으로 생성/파괴.
`ObjectStorageSlotUI`	`Game.ObjectSystem.UI` (Managers)	보관소 내 개별 아이템 슬롯 컴포넌트. 클릭 시 `PlacementManager.Instance.SelectPlacementType()` 을 호출하여 **즉시 배치 모드로 진입하고 보관소 창을 닫음**.

2.2 ObjectDropManager의 동적 드랍 확률 공식

웨이브 종료 시까지 플레이어가 목표한 개수의 오브젝트를 안정적으로 획득할 수 있도록 동적 가중치 알고리즘을 사용합니다.

> **동적 드랍 확률 계산 알고리즘**
>
> `남은 적 수(Remaining Enemies) = totalEnemiesInWave - (killedEnemyCount - 1)`
> `남은 필요 드랍 수(Remaining Drops) = targetDropCount - currentDroppedCount`
> • 만약 `Remaining Enemies <= Remaining Drops` 라면: **100% 확정 드랍 (Probability = 1.0)**
> • 그 외: `Probability = Remaining Drops / Remaining Enemies`
> • *적 베이스 도달 시:* `killedEnemyCount` 만 증가시키고 드랍 기회는 영구 소멸

3 오브젝트 배치 (Placement) 시스템 구조

배치 시스템은 정비 시간(`MaintenanceTime`)에 플레이어가 보관소의 오브젝트를 전장에 자유롭게 청사진(Blueprint) 형태로 배치, 회전, 확정하거나 기존 설치물을 재조정/회수할 수 있도록 지원합니다.

3.1 핵심 클래스 명세

클래스명	파일 경로	책임 및 주요 역할
`PlacementManager`	`Assets/Game/Scripts/Managers`	**배치 파이프라인 총괄 싱글톤**. 청사진 생성, 레이캐스트 바닥 투영, 장애물/여유반경(Buffer) 겹침 검사(`IsPlacementValid()`), 임시 고정 및 3버튼 UI 표시, 설치 확정(실물화) 및 보관소 차감, 회수 처리.
`PlacementControlUI`	`Assets/Game/Scripts/Placement`	청사진 상단 3D 월드 좌표에 오프셋(`uiOffset`)을 더해 스크린 좌표로 띄워지는 3버튼 인터페이스. **[설치 확정] / [회전 (45°)] / [회수]** 이벤트 바인딩.
`SlingshotSet`	`Assets/Game/Scripts/Placement`	필드에 설치되는 슬링샷 복합체. 좌/우 범퍼 기둥 트랜스폼을 기반으로 고무줄 스크립트(`SlingshotRubber`)의 양 끝단 앵커를 실시간 동기화.
`SlingshotRubber`	`Assets/Game/Scripts/Placement`	슬링샷 고무줄 물리 본체. `LineRenderer` 3포인트와 `BoxCollider` 를 동적 계산. 펭귄 충돌 시 전방 고정 임펄스 속도(`launchForce`) 부여 및 감쇄 진동(Damped Wave) 연출 수행.
`BumperBounceSurface`	`Assets/Game/Scripts/Physics`	필드에 설치되는 범퍼. `PinballBounceSurface` 를 상속받아 펭귄 충돌 시 입사각과 반사 계수, 마찰력을 적용해 반발 속도 부여 및 콤보 증가.

3.2 PlacementManager의 청사진 안전 제어 메커니즘

청사진(Blueprint) 배치 중 카메라 레이저 충돌 및 물리 오류를 방지하기 위해 4중 안전 처리가 적용됩니다.

1. LayerMask 격리

청사진의 모든 자식 트랜스폼 레이어를 `Ignore Raycast` 로 임시 치환하여 바닥 레이캐스트가 자기 자신을 감지해 카메라 쪽으로 끌려오는 현상 원천 차단.

2. 물리/콜라이더 안전화

모든 Rigidbody를 `isKinematic = true` , 속도 0으로 잠금. Collider는 `isTrigger = true` 로 전환하여 다른 물리 오브젝트를 밀쳐내지 않도록 방지.

3. MonoBehaviour 스크립트 비활성화

청사진 상태에서 슬링샷 발사, 범퍼 반발 등의 기능 스크립트가 실행되지 않도록 모든 컴포넌트 `enabled = false` 처리.

4. 시각적 Validation 피드백

`Physics.OverlapBox` 로 여유 버퍼(`bufferRadius`) 포함 장애물 겹침 여부를 검사하여 설치 가능 시 초록색 반투명, 불가능 시 빨간색 반투명 렌더링.

4 엔드-투-엔드 전체 라이프사이클 흐름

3대 시스템이 유기적으로 맞물려 실행되는 3가지 대표 시나리오의 단계별 호출 흐름입니다.

시나리오 1: 적 처치 ➔ 오브젝트 획득 및 보관소 입고

1. 펭귄 충돌 ➔ `EnemyBounceSurface.OnHitImpact()` 실행 ➔ `CombatResolver.DealDamage()`로 피해 적용

2. 적 사망 ➔ HP가 0이 되어 `NormalEnemyAI.Die()` 호출 ➔ `ObjectDropManager.OnEnemyKilledByPlayer()` 호출

3. 드랍 연산 ➔ 동적 확률 계산 통과 시 `new ObjectItem(selectedType)` 생성 ➔ `ObjectStorageManager.AddObject()` 호출

4. UI 갱신 ➔ `OnObjectAdded` 이벤트 발생 ➔ `ObjectStorageUI`가 `ObjectStorageSlotUI`를 동적 인스턴스화

시나리오 2: 보관소 선택 ➔ 필드 청사진 배치 ➔ 설치 확정

1. 슬롯 클릭 ➔ `ObjectStorageSlotUI.OnPointerClick()` ➔ `PlacementManager.SelectPlacementType()` 호출 후 창 닫기

2. 청사진 생성 ➔ `PlacementManager.CreateBlueprint()` (Ignore Raycast 설정, 물리/스크립트 비활성화, 마우스 추적)

3. 위치 고정 ➔ 유효 위치 바닥 좌클릭 ➔ `isFixedPosition = true` ➔ 머리 위에 `PlacementControlUI` 표시

4. 설치 확정 ➔ [확정] 클릭 ➔ 레이어/물리/스크립트 복원 (실물화) ➔ `ObjectStorageManager.RemoveObject()`로 차감

시나리오 3: 맵 전환 (씬 로드) 시 일괄 초기화

1. 씬 전환 감지 ➔ `SceneManager.sceneLoaded` 이벤트 발생 ➔ `ObjectStorageManager.OnSceneLoaded()` 수신

2. 보관소 정리 ➔ `ObjectStorageManager.ClearOnMapTransition()` 실행 ➔ `_storedObjects.Clear()`

3. 필드 정리 ➔ `PlacementManager.ClearAllPlacedObjects()` 연동 ➔ 필드 설치물 및 청사진 전량 Destroy

> 📌 **요약 및 유지보수 가이드라인**
>
> 새로운 필드 오브젝트(예: 포털, 텔레포터 등)를 추가할 때는 **1)** `ObjectType` 및 `PlacementType` 열거형에 추가, **2)** `ObjectDropManager.availableTypes`에 타입 등록, **3)** `PlacementManager`에 프리팹 등록, **4)** 오브젝트 프리팹에 충돌 및 기능 컴포넌트(`PinballBounceSurface` 상속 권장)를 부착하면 기존 보관소-배치 파이프라인에 완벽하게 통합됩니다.